

## Sprint 2 — SQL Server, Entity Framework Core, ASP.NET MVC & WebAPI

---

**Duration:** Week 3 – Week 4 (2 weeks)

**Curriculum:** Databases (Wk 3) | ASP.NET MVC + EF Core + WebAPI (Wk 4)

**App Review 2:** Viva + Project — CRUD API Service (End of Week 4)

**PS-02 Deliverable:** An ASP.NET MVC web app + REST WebAPI — full CRUD for patients, doctors, appointments, and health records backed by SQL Server

### Learning Objectives

- Design a normalised relational database schema — tables, primary keys, foreign keys, indexes.
- Write SQL queries — SELECT, INSERT, UPDATE, DELETE, JOINS, and stored procedures.
- Use Entity Framework 6.x and EF Core with Code First — DbContext, entity configurations, and migrations.
- Implement the Repository Pattern and Unit of Work to keep business logic decoupled from data access.
- Use MongoDB for storing audit logs and reference data (specialisations, appointment types).
- Build an ASP.NET Core MVC application — controllers, Razor views, model binding, and routing.
- Apply server-side validation using Data Annotations — Required, StringLength, Range, RegularExpression.
- Build a parallel REST WebAPI — HTTP verbs, status codes, Swagger documentation, and Dependency Injection.

### Database Schema

**Patients:** PatientId (PK, int, identity), FullName (nvarchar 100), DateOfBirth (date), Gender (nvarchar 10), PhoneNumber (nvarchar 20), Email (nvarchar 100, unique), InsuranceId (nvarchar 50), CreatedDate (datetime2)

**Doctors:** DoctorId (PK, int, identity), FullName (nvarchar 100), Specialisation (nvarchar 50), YearsOfExperience (int), ConsultationFee (decimal 10,2), IsActive (bit)

**Appointments:** AppointmentId (PK, int, identity), PatientId (FK → Patients), DoctorId (FK → Doctors), ScheduledDate (date), TimeSlot (nvarchar 20), Status (nvarchar 20), CancellationReason (nvarchar 500 null)

**HealthRecords:** RecordId (PK, int, identity), PatientId (FK → Patients), DoctorId (FK → Doctors), VisitDate (datetime2), Diagnosis (nvarchar 500), Prescription (nvarchar 500), Notes (nvarchar 1000 null)

**Users:** UserId (PK, int, identity), Email (nvarchar 100, unique), PasswordHash (nvarchar 256), Role (nvarchar 20 — Patient / Doctor / Admin), ReferenceId (int — links to PatientId or DoctorId)

### What Trainees Will Build

An ASP.NET Core MVC web application and a parallel REST WebAPI, both connected to the same SQL Server database via EF Core. The domain classes from Sprint 1 are evolved into EF Core entities. The MVC app provides Razor-view-based screens. The WebAPI exposes the same operations as JSON endpoints documented in Swagger.

**EF Core DbContext & Migrations:** AppDbContext configured with DbSet<Patient>, DbSet<Doctor>, DbSet<Appointment>, DbSet<HealthRecord>, DbSet<User>. Entity relationships configured (one Patient → many Appointments, one Doctor → many Appointments, one-to-many HealthRecords). Initial migration created, database seeded with 5 doctors across different specialisations and 3 sample patients.

**Repository Pattern:** IRepository<T> (generic) with GetById, GetAll, Add, Update, Delete. Concrete repositories: PatientRepository, DoctorRepository, AppointmentRepository, HealthRecordRepository. IUnitOfWork with Commit() — controllers never call DbContext directly.

**Patient Management (MVC):** Pages: Patient List (sortable by name, filterable by insurance status), Patient Register form (all fields with validation, duplicate email check), Patient Profile view (read-only summary with appointment count), Patient Edit form, Patient Deactivate confirmation. All validation errors displayed inline in the Razor view.

**Doctor Management (MVC):** Pages: Doctor List (filterable by specialisation, sorted by name), Add Doctor form (all fields, fee must be positive), Doctor Profile view (full details + upcoming appointment count), Edit Doctor form, Toggle Active/Inactive status.

**Appointment Booking (MVC):** Booking page: patient selects a doctor from a dropdown, picks a date (calendar picker, no past dates), selects from available time slots. On submit, slot availability is checked before saving. Duplicate-slot validation with a clear error message if the slot is already taken.

**Appointment Management (MVC):** Patient view: "My Appointments" — list of all appointments with status badges. Doctor view: "Today's Schedule" and "Weekly Schedule" — all appointments on a given date. Status update: Doctor can mark an appointment as Confirmed or Cancelled (with a reason) from the list view.

**Health Record Entry (MVC):** After marking an appointment as Completed, the doctor is presented with a "Add Health Record" form. Fields: Diagnosis, Prescription, Notes. Submitted record is linked to the appointment, patient, and doctor.

**Patient Health History (MVC):** A read-only Razor page listing all health records for a patient, ordered by VisitDate descending. Each record shows doctor name, specialisation, visit date, diagnosis, and prescription on a single row.

**REST WebAPI (parallel):** Swagger-documented endpoints: GET /api/patients, GET /api/patients/{id}, POST /api/patients, PUT /api/patients/{id}. GET /api/doctors, GET /api/doctors/{id}, GET /api/doctors?specialisation={s}. GET /api/appointments, POST /api/appointments, PUT /api/appointments/{id}/status, DELETE /api/appointments/{id}. GET /api/health-records/{patientId}, POST /api/health-records. All endpoints return appropriate HTTP status codes (200, 201, 400, 404).

## **App Review 2 — Assessment Focus**

- Database schema is correctly normalised — all foreign keys present, no redundant columns.
- EF Core Code First migrations run cleanly — database created from scratch with one command.
- Repository pattern is correctly implemented — no DbContext calls directly in controllers.
- All MVC forms have server-side validation — required fields, format checks, and error messages visible.
- Appointment booking validates slot availability — double-booking returns a clear error, not an exception.
- WebAPI returns correct HTTP status codes — 200, 201, 400, 404 used appropriately.
- Swagger is accessible at /swagger and all API endpoints are described.
- Application runs from scratch with migrations applied and all operations work end-to-end.